

Nexus ERP Fase 4

Para que o seu sistema comece a ganhar “vida” visualmente, vamos criar uma página **Frontend** simples e moderna. Ela usará HTML, CSS e JavaScript puro (sem precisar instalar nada extra) para se comunicar com o seu Backend Python.

Mas antes, precisamos fazer um pequeno ajuste no seu **Backend** (`main.py`) para permitir que o navegador aceite a conexão (o que chamamos de **CORS**).

Passo 1: Ajuste no Backend (`main.py`)

Adicione estas linhas ao seu arquivo `main.py` para que o Frontend consiga “conversar” com o Backend.

```
```python
```

```
from fastapi.middleware.cors import CORSMiddleware
```

```
... (mantenha o código anterior) ...
```

```
App = FastAPI(title="Nexus ERP Real-Time")
```

```
ADICIONE ISSO AQUI:
```

```
App.add_middleware(
```

```
 CORSMiddleware,
```

```
 Allow_origins=["*"], # Permite que qualquer página acesse (Ideal para teste)
```

```
 Allow_methods=["*"],
```

```
 Allow_headers=["*"],
```

```
)
```

```
... (resto do código dos endpoints) ...
```

...

---

### ### Passo 2: O Frontend (`index.html`)

Crie um arquivo chamado `index.html` na mesma pasta. Usei um estilo “Clean Microsoft” para dar aquela cara de **Praticidade**.

```
```html<!DOCTYPE html><html lang="pt-br"><head>  <meta charset="UTF-8">  <title>Nexus ERP – Painel de Vendas</title>  <style>    Body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;    background: #f4f7f6; margin: 40px; }    .card { background: white; padding: 25px; border-radius: 8px; box-shadow: 0    4px 6px rgba(0,0,0,0.1); max-width: 500px; margin-bottom: 20px; }    H2 { color: #2c3e50; margin-top: 0; }    Input { width: 100%; padding: 10px; margin: 10px 0; border: 1px solid #ddd;    border-radius: 4px; box-sizing: border-box; }    Button { background: #0078d4; color: white; border: none; padding: 10px 20px;    border-radius: 4px; cursor: pointer; width: 100%; font-weight: bold; }    Button:hover { background: #005a9e; }    Table { width: 100%; border-collapse: collapse; background: white; border-    radius: 8px; overflow: hidden; box-shadow: 0 4px 6px rgba(0,0,0,0.1); }    Th, td { padding: 12px; text-align: left; border-bottom: 1px solid #eee; }    Th { background: #0078d4; color: white; }  </style></head>
```

```
<body>
```

```
<div class="card">
```

```
<h2>Nova Venda (Nexus ERP)</h2>
```

```
<input type="text" id="customer" placeholder="Nome do Cliente">
```

```
<input type="number" id="value" placeholder="Valor da Venda (R$)">
```

```
<button onclick="salvarVenda()">Registrar no Banco Real</button>
```

```
</div>
```

```
<h2>Vendas Registradas</h2>
```

```
<table id="tabelaVendas">
```

```
<thead>
```

```
<tr>
```

```
<th>ID</th>
```

```
<th>Cliente</th>
```

```
<th>Valor (R$)</th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
<!--As vendas aparecerão aqui -->
```

```
</tbody>
```

```
</table>
```

```
<script>
```

```
Const API_URL = http://localhost:8000;
```

```
// Função para salvar a venda no Backend
```

```
Async function salvarVenda() {
```

```
Const customer = document.getElementById('customer').value;
```

```
Const value = document.getElementById('value').value;
```

```
Const response = await  
fetch(`${API_URL}/sales/new?customer=${customer}&value=${value}`, {  
  Method: 'POST'  
});
```

```
If (response.ok) {  
  Alert("Venda gravada no PostgreSQL com sucesso!");  
  Document.getElementById('customer').value = '';  
  Document.getElementById('value').value = '';  
  carregarVendas(); // Atualiza a lista  
} else {  
  Alert("Erro ao gravar venda.");  
}  
}
```

```
// Função para carregar as vendas do banco (opcional: criar endpoint /sales no  
backend para listar)
```

```
Async function carregarVendas() {  
  // Aqui você chamaria um GET /sales que criamos no main.py  
}  
</script>  
</body>  
</html>  
...
```

Como Testar o Ciclo Completo (Front -> Back -> Banco):

1. **Ligue o Banco:** No terminal, rode ``docker-compose up -d``.
2. **Ligue o Backend:** No terminal, rode ``uvicorn main:app --reload``.
3. **Abra o Frontend:** Clique duas vezes no arquivo ``index.html``.
4. **Ação:** Digite um nome e um valor, e clique no botão.

* **O que acontece:** O JavaScript envia o dado para o Python. O Python valida e grava no PostgreSQL. O banco confirma e você recebe o alerta.

Por que isso é um marco para o seu projeto?

* **Fim da Teoria:** Agora você tem um sistema real. Se você digitar uma venda agora, fechar tudo, reiniciar o computador e olhar no banco, **a venda estará lá**.

* **Qualidade Visual:** O formulário é simples, mas segue o padrão de **Praticidade**.

* **Fundação de ERP:** Você pode duplicar esse ``index.html`` para criar a página de **Compras** e a página de **RH**, usando a mesma lógica.

Próximo Passo:

Você quer que eu adicione a funcionalidade de **”Listar Vendas”** (para que elas apareçam na tabela automaticamente assim que você carregar a página)? Isso daria o toque final de **“Sistema Vivo”**.